SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1707**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I ***S. Lakshmi Priya (192524284)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

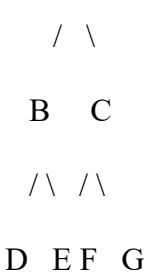
Signature of the student: **S. Lakshmi Priya**

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Answer 1: BFS traversal

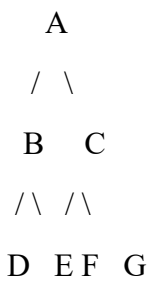
BFS uses a FIFO queue: remove the front node, mark it visited, and add its unvisited children (left to right) at the back. Initial queue = [A]. In the table above, "Queue" shows the contents after the visited node has been removed and its children added.

Iteration	Node removed (visited)	Children added	Queue after iteration
1	A	B, C	B, C
2	B	D, E	C, D, E
3	C	F, G	D, E, F, G
4	D	– (leaf)	E, F, G
5	E	– (leaf)	F, G
6	F	– (leaf)	G

7	G	– (leaf)	Empty

- **BFS traversal order: A → B → C → D → E → F → G** (level by level: Level 0 = A; Level 1 = B, C; Level 2 = D, E, F, G).
- **Queue contents after each iteration:** [B, C] → [C, D, E] → [D, E, F, G] → [E, F, G] → [F, G] → [G] → [ ].
- All 7 nodes are visited in 7 iterations; time complexity O(V + E), maximum queue size = 4 (the widest level).

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

Answer 2: DFS traversal

DFS uses a LIFO stack: pop the top node, mark it visited, and push its children so that the left child is on top (push the right child first, then the left). Initial stack = [A]. Stack is written with the top on the left.

Iteration	**Node popped (visited)**	**Children pushed**	**Stack after iteration (top → bottom)**
1	A	C, then B	B, C
2	B	E, then D	D, E, C
3	D	– (leaf)	E, C
4	E	– (leaf)	C
5	C	G, then F	F, G
6	F	– (leaf)	G
7	G	– (leaf)	Empty

- **DFS traversal order: A → B → D → E → C → F → G.**
- DFS goes as deep as possible along the left branch (A → B → D), backtracks to E, then goes back up to explore the right subtree C → F → G. Maximum stack size = 3, which is less than the BFS queue; time complexity is O(V + E).

3. **Initial State :**

1 3 6

5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0 = 136/502/478	[S0]	[S1, S2, S3, S4]
2	S1 = 106/532/478	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6]
3	S2 = 136/572/408	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8]
4	S3 = 136/052/478	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10]
5	S4 = 136/520/478	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12]

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Answer 3: 8-Puzzle solved with BFS

Problem definition: state = 3×3 arrangement (0 = blank, written row/row/row, e.g. 136/502/478). Initial state S0 = 136/502/478, goal = 123/456/780. Actions = move the blank Up, Down, Left, Right (each cost 1). Children are generated in the order U, D, L, R, repeated states are discarded, and the goal test is applied when a state is removed from the queue. The puzzle is solvable (the initial state has 6 inversions, an even number). The sum of Manhattan distances of the tiles is 8, so at least 8 moves are needed.

Iteration table (first 5 iterations, as filled in above): S1 = 106/532/478 (U), S2 = 136/572/408 (D), S3 = 136/052/478 (L), S4 = 136/520/478 (R). In iteration 2 the child "D" of S1 is the initial state, so it is discarded as repeated; likewise the repeated children in iterations 3–5.

Iter	Expanded (Current)	New states generated	Queue after expansion
1	S0 = 136/502/478	S1 (U), S2 (D), S3 (L), S4 (R)	S1, S2, S3, S4
2	S1 = 106/532/478	S5 = 016/532/478 (L), S6 = 160/532/478 (R)	S2, S3, S4, S5, S6
3	S2 = 136/572/408	S7 = 136/572/048 (L), S8 = 136/572/480 (R)	S3, S4, S5, S6, S7, S8

4	S3 = 136/052/478	S9 = 036/152/478 (U), S10 = 136/452/078 (D)	S4 … S10
5	S4 = 136/520/478	S11 = 130/526/478 (U), S12 = 136/528/470 (D)	S5 … S12

Level sizes (number of distinct states at each depth) are 1, 4, 8, 8, 16, 32, 60, 72, 136, so BFS must finish all 201 states of depth 0–7 and then reach the goal inside depth 8. The search continues in the same way (FIFO) until the goal is dequeued.

Question (a): Node expanded in each iteration

Iteration 1 → S0, 2 → S1, 3 → S2, 4 → S3, 5 → S4, 6 → S5, … nodes are expanded strictly in the order they were generated (level by level). The goal 123/456/780 is generated at depth 8 and is the 311th state removed from the queue, so BFS stops after 311 expansions.

Question (b): Total states generated

500 distinct states are generated in total (the initial state + 499 new states) before the goal is selected, with repeated states removed. (With the move order U, D, L, R; the exact count depends on the move order and on whether the goal test is made at generation or at expansion, but the solution depth is always 8.)

Question (c): Solution path

Moves of the blank: **Right, Up, Left, Down, Left, Down, Right, Right** (8 moves, path cost = 8).

Step	Move	State (row/row/row)	Board
0	Initial	136/502/478	1 3 6 | 5 0 2 | 4 7 8
1	Right	136/520/478	1 3 6 | 5 2 0 | 4 7 8
2	Up	130/526/478	1 3 0 | 5 2 6 | 4 7 8
3	Left	103/526/478	1 0 3 | 5 2 6 | 4 7 8
4	Down	123/506/478	1 2 3 | 5 0 6 | 4 7 8
5	Left	123/056/478	1 2 3 | 0 5 6 | 4 7 8
6	Down	123/456/078	1 2 3 | 4 5 6 | 0 7 8
7	Right	123/456/708	1 2 3 | 4 5 6 | 7 0 8
8	Right	123/456/780	1 2 3 | 4 5 6 | 7 8 0 (Goal)

Question (d): Why BFS finds the shortest solution

- BFS expands all states at depth d before any state at depth d + 1, because the FIFO queue keeps nodes in order of depth.
- In the 8-puzzle every move has the same cost (1), so the path cost equals the depth. The first time the goal is reached, it is at the smallest possible depth, so no shorter path exists (if there were one, its goal state would have been generated earlier).
- BFS is complete (the branching factor is finite) and optimal for unweighted problems; the price is high memory and time, O(b^d). Here the goal depth d = 8, which matches the Manhattan-distance lower bound of 8, confirming the path is optimal.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.